

# Test Strategy Analysis

## Part 3 — Test Strategy & Scenario Mining (Coupon Discount Calculation)

- Date: January 30, 2026
- Role: Software Test Engineer (QA Automation) — Case Study Deliverable
- Author: Husnuye (Candidate)
- Repository deliverable: Playwright (TypeScript) API + UI automation + visual testing

### Problem statement

Customers on an e-commerce site report: “The total discount is calculated incorrectly when a coupon code is used.”

Goal: use an LLM as a partner to explore edge cases, then apply QA judgment to select the most critical scenarios and refine a final Gherkin scenario.

- Focus: pricing correctness, rule interactions, rounding, and UI/backend consistency.
- Outputs: scenario list, prioritized top 5, QA notes, and final Gherkin scenario.

Note: Automation implementation for this case study is provided separately in the repository (API lifecycle tests, UI checkout flow, and visual regression). CI publishes Playwright HTML reports via GitHub Pages for easy review.

# 1. Test strategy overview

## Objectives

- Detect incorrect discount totals under coupon usage, especially where multiple pricing rules interact.
- Prevent revenue loss or customer-impacting under/over-discount.
- Ensure consistent totals across UI, backend invoice, and payment capture.

## Scope

- Coupon types: percentage and fixed amount; shipping and tax interactions.
- Eligibility rules: specific items/categories/brands; mixed carts.
- Caps/minimum thresholds; rounding rules; state changes after applying coupon.

## Test types

- Service-level tests for pricing engine (contract + calculation assertions).
- End-to-end UI checkout tests verifying displayed totals and final confirmation.
- Visual regression checks for key inventory screen differences (where applicable).

## Key risks / failure modes

- Wrong order-of-operations (item promos vs coupon vs tax/shipping).
- Incorrect allocation of coupon across eligible items; rounding drift.
- Negative totals or exceeding maximum discount cap.
- State transition bugs: quantity edits, item removal, coupon replacement.

## Test data strategy

- Use representative carts: mixed eligibility, promo vs non-promo items, boundary prices.
- Include boundary values for min-subtotal and max-cap thresholds.
- Validate money formatting and standard rounding to 2 decimals.

## 2. Scenario mining

### 2.1 20 edge case scenarios (LLM-generated, reviewed)

- Percentage coupon + existing item-level promotion: order of operations causes double-discount or wrong totals.
- Fixed-amount coupon exceeds cart subtotal → negative total or below-zero discount.
- Coupon applies only to certain categories; mixed cart contains eligible + ineligible items.
- Coupon applies only to specific SKUs/brands; mixed cart distribution is incorrect.
- Coupon requires minimum subtotal; applying coupon drops subtotal below threshold (re-check logic).
- Coupon has maximum discount cap; cap not enforced or enforced incorrectly.
- Free-shipping coupon combined with percentage coupon: shipping discount counted incorrectly in “total discount”.
- Single-use coupon reused by same user/session; partial discount still applied.
- First-order-only coupon: guest vs registered vs returning customer mis-detection.
- Tax-inclusive vs tax-exclusive calculation: discount applied to wrong base.
- Shipping included vs excluded from discount base: “subtotal” vs “total” ambiguity.
- Removing items/changing quantity after applying coupon: recalculation drift.
- Bundles/kits: coupon allocation across bundle items wrong.
- Replacing coupon A with coupon B: stale discount remains or stacked unexpectedly.
- Rounding issues: 0.01 differences due to per-line rounding vs total rounding.
- Multi-currency/cart currency conversion: coupon value converted incorrectly.
- Quantity limits: coupon applies only to N items; extra quantity miscounted.
- Promo price already applied (sale price) + coupon: discount double-counted.
- Gift card + coupon: application order causes wrong “total discount”.
- Split shipment / multi-seller: coupon should apply to subset but applied globally.

## 2.2 Prioritization and QA review notes

### Top 5 most critical scenarios (selected)

- % coupon + existing item promotion (wrong order / double-discount) — common setup; direct revenue & trust impact.
- Fixed-amount coupon > subtotal (negative total / exploit) — fraud risk; payment/ledger failures.
- Eligible-items-only coupon on a mixed cart (allocation) — frequent bug source; needs correct pro-rating + transparency.
- Min subtotal + max discount cap together (rule engine complexity) — over/under discount if misapplied.
- Rounding rules (line vs total rounding) — small differences trigger disputes; UI/invoice mismatch.

### QA gaps AI often misses

- Discount allocation strategy (pro-rate across items) and how rounding affects totals.
- Explicit order-of-operations: item promos → coupon → tax → shipping (or other).
- State transitions: removing items, quantity updates, switching coupons must re-evaluate correctly.
- UI vs backend consistency: checkout UI totals must match invoice/payment totals.
- Concurrency/idempotency: duplicate apply requests can double-apply discount.

### Most complex scenario (from Top 5)

The most complex is the combined-constraints scenario: mixed cart + existing item promo + percent coupon + max cap + minimum threshold + rounding. This is where rule engines commonly fail and where we should place the highest-signal end-to-end assertions.

### 3. Final Gherkin scenario (refined)

This Gherkin is written to be implementation-ready: clear preconditions, explicit calculation rules, and verifiable outcomes.

Feature: Coupon discount calculation

Background:

Given the pricing service is available  
And the customer is on the checkout page

Scenario: Correct total discount for mixed cart with item promotion, coupon cap and minimum threshold

Given the cart contains:

| sku   | name           | qty | unit_price | item_promo_type | item_promo_value |
|-------|----------------|-----|------------|-----------------|------------------|
| A-001 | Running Shoes  | 1   | 120.00     | PERCENT         | 10               |
| B-010 | Socks (3-pack) | 2   | 25.00      | NONE            | 0                |

And shipping fee is 10.00

And tax is calculated on discounted item totals

And a coupon code "SAVE20" is configured as:

| type                | value | applies_to          | min_subtotal | max_discount |
|---------------------|-------|---------------------|--------------|--------------|
| PERCENT_ON_SUBTOTAL | 20    | ELIGIBLE_ITEMS_ONLY | 100.00       | 30.00        |

When the customer applies the coupon code "SAVE20"

Then item-level promotions should be applied before the coupon discount

And the coupon discount should apply only to eligible items after item-level promotions

And the coupon discount should not exceed the maximum discount cap of 30.00

And the subtotal after all discounts should be greater than or equal to 0.00

And the total discount displayed should equal the sum of item-level promotions and coupon discount

And all monetary values should be rounded to 2 decimal places using standard rounding

And the order total displayed should match the backend invoice total

Review note: In the repository CI, Playwright HTML reports are published to GitHub Pages (API: /apitests/, UI: /webtests/) and artifacts are uploaded for debugging.